

miniFlink: руководство для начала работы

Сквозной разбор от потока-функции до durable-пайплайна

Проект miniFlink

Аннотация

Это пошаговое руководство для тех, кто впервые видит miniFlink. Мы строим один пайплайн от начала до конца на сквозном примере — телеметрии температурных датчиков — и на каждом шаге добавляем ровно одно понятие: поток, событие и время, watermark, ключ группировки, окна, агрегацию, stateful-правила, дедупликацию и, наконец, персистентность. Все листинги взяты из компилируемого кода. Прочитав руководство сверху вниз, вы сможете читать и писать пайплайны miniFlink и будете знать, куда смотреть дальше. Примеры по возрастанию сложности живут в каталоге `examples/` (`ex01–ex10`); это руководство — сквозная нить, связывающая их.

Содержание

1	Прежде чем начать	1
2	Шаг 1. Поток как функция	1
3	Шаг 2. Событие и время	2
4	Шаг 3. Watermarks	2
5	Шаг 4. Ключ группировки через KEYED	3
6	Шаг 5. Окна и агрегация	3
7	Шаг 6. Stateful-правила: триггеры	3
8	Шаг 7. Дедупликация поздних дублей	4
9	Шаг 8. Поздние данные: retract и update	4
10	Шаг 9. Персистентность без изменения пайплайна	4
11	Шаг 10. Масштабирование и exactly-once	5
12	Что дальше	6

1 Прежде чем начать

miniFlink — это библиотека потоковой обработки на OCaml, реализующая ядро семантики Apache Flink на одном узле: событийное время, watermarks, окна, stateful-операторы, retract/update для поздних данных и exactly-once. Никакого брокера, планировщика или фонового цикла; весь пайплайн — это композиция обычных функций, которую можно читать как спецификацию.

Чтобы запускать примеры, нужен OCaml (4.14+ или 5.x) и Dune 3.0+. Соберите проект командой `dune build` и прогоните тесты — `dune test`. Каждый листинг ниже предполагает, что в начале файла стоит `open Miniflink`.

Сквозной пример. На протяжении всего руководства мы обрабатываем поток показаний температурных датчиков и хотим в итоге: считать среднюю температуру по датчику в окне, поднимать тревогу при перегреве и переживать перезапуск процесса без потери состояния. Доменная модель проста:

```
type reading = { sensor : string; celsius : float }
```

2 Шаг 1. Поток как функция

Центральная абстракция — `Stream.t`, и она устроена предельно просто:

```
type 'a t = unit -> 'a option
```

Поток — это функция без аргументов, которая при каждом вызове возвращает либо следующий элемент (`Some x`), либо `None`, если элементы кончились. Это *pull*-модель: ничего не происходит, пока потребитель не «потянет» следующее значение. Весь граф обработки — это стопка таких функций, обёрнутых одна вокруг другой.

```
let nats : int Stream.t =
  let i = ref 0 in
  fun () -> let v = !i in incr i; if v < 5 then Some v else None

let doubled = Stream.map (fun x -> x * 2) nats
```

`Stream.map`, `Stream.filter` и `Stream.flat_map` — это операторы преобразования: каждый принимает поток и возвращает новый поток, не вычисляя ничего заранее. Композиция через оператор `|>` читается естественно слева направо.

3 Шаг 2. Событие и время

Голый поток значений ничего не знает о времени. В потоковой обработке время — первоклассное понятие, поэтому `miniFlink` оборачивает значения в `Mf_event.t` с четырьмя вариантами:

- `Data (value, ts)` — значение с временной меткой (event-time);
- `Watermark ts` — отметка прогресса времени (см. шаг 3);
- `Retract (value, ts)` — отмена ранее выданного результата;
- `Update {old; new_value; ts}` — атомарная замена старого результата новым.

Последние два понадобятся, когда мы дойдём до поздних данных; пока что работаем с `Data`. Превратим наши показания в event-time поток — пусть каждое следующее чтение приходит на секунду позже:

```
let event_stream : reading Mf_event.t Stream.t =
  readings
  |> List.mapi (fun i r -> Mf_event.data r (i * 1000))
  |> Stream.of_list
```

Временные метки здесь в миллисекундах; `Time` — это просто `int` мс с удобными конструкторами (`Time.seconds`, `Time.minutes`).

4 Шаг 3. Watermarks

Событийное время порождает вопрос: когда можно считать, что «все события до момента t уже пришли»? В распределённой системе данные опаздывают и приходят не по порядку. *Watermark* — это обещание системы: «событий с меткой меньше t больше не будет». Операторы окон используют его, чтобы понять, когда окно можно закрывать и эмитить результат.

```
let with_wm =  
  Mf_event.with_watermarks ~latency:(Time.seconds 1) event_stream
```

Параметр `~latency` задаёт допуск на опоздание: watermark отстаёт от максимальной увиденной метки на эту величину, что оставляет опоздавшим событиям шанс прийти. Чем больше `latency`, тем устойчивее к опозданиям, но тем позже закрываются окна — это прямой компромисс между полнотой и задержкой.

5 Шаг 4. Ключ группировки через KEYED

Почти все интересные операторы работают *по ключу*: окно «на датчик», агрегат «на пользователя». В большинстве фреймворков ключ передаётся параметром в каждый оператор. miniFlink кодирует ключ *один раз* в типе через first-class-модуль `Keyed.S`:

```
module By_sensor : Keyed.S with type t = reading = struct  
  type t = reading  
  let key r = r.sensor  
end
```

Теперь `By_sensor` можно передавать операторам как значение, и им не нужен отдельный параметр `~key`. Для разовых случаев есть `Keyed.of_fun (fun r -> r.sensor)`, не требующий объявления модуля. Этот приём и делает пайплайны похожими на спецификацию: ключ виден один раз, а не повторяется в каждом вызове.

6 Шаг 5. Окна и агрегация

Окно режет бесконечный поток на конечные куски, к которым можно применить агрегат. Простейшее — *тумблинг*: неперекрывающиеся окна фиксированного размера. Посчитаем среднюю температуру по каждому датчику в окнах по три секунды:

```
let avg_per_sensor =  
  event_stream  
  |> Pipe.window_agg (module By_sensor)  
    (Pipe.tumbling (Time.seconds 3))  
    (Agg.mean (fun r -> r.celsius))
```

`Pipe.window_agg` принимает модуль ключа, спецификацию окна (`tumbling`, `sliding`, и т. д.) и *агрегат*. Агрегаты — это композируемая библиотека: `count`, `sum`, `mean`, `count_if`, а также комбинаторы вроде `both` (считать два агрегата сразу) и `contramap` (адаптировать входной тип). На выходе — поток (`string * 'r`): ключ и результат агрегата за закрытое окно. Результат появляется, когда watermark пересекает границу окна — вот зачем был нужен шаг 3.

7 Шаг 6. Stateful-правила: триггеры

Окна отвечают на «сколько/какое среднее», но для системы оповещения нужен *stateful*-автомат: поднять тревогу, когда значение пересекает порог, и снять её, когда вернулось в норму, не дёргая оператора на каждом событии. Это **Trigger**:

```
let alert_spec =
  Trigger.create
    ~name:"overheat"
    ~condition:(Trigger.greater_than 90.0)
    ~produce_alert:(fun ~key ~value ~ts:_ ->
      Printf.sprintf "ALERT %s=%.0f" key value)
    ~produce_recovery:(fun ~key ~ts:_ ->
      Printf.sprintf "OK %s" key)
    ()

let alerts =
  event_stream
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Trigger.of_stream alert_spec
```

Триггер — это конечный автомат на ключ. Когда температура датчика превышает 90, он эмитит **Data** с алертом; когда падает обратно — **Retract** прежнего алерта и **Data** с recovery. Можно задать debounce (**~problem_for**), гистерезис (**greater_than_with_hysteresis**) и произвольные условия (**custom**). Важное свойство, проверенное property-тестами: активных алертов на ключ не больше одного, а каждый recovery парен своему алерту — автомат не «залипает».

8 Шаг 7. Дедупликация поздних дублей

Семантика **at-least-once** у источников означает дубли: один пакет может прийти дважды. **Pipe.dedup** подавляет повторы с одним ключом в пределах окна **cooldown**:

```
module By_pair : Keyed.S with type t = string * float = struct
  type t = string * float
  let key (k, _) = k
end

let deduped =
  event_stream
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Pipe.dedup (module By_pair)
    ~rule:(fun (k, _) -> k)
    ~cooldown:(Time.seconds 5)
```

Свойство, зафиксированное property-тестом: между двумя прошедшими событиями одного ключа разрыв по времени не меньше **cooldown**, и каждое прошедшее событие действительно было во входе.

9 Шаг 8. Поздние данные: retract и update

Вернёмся к четырёхвариантному событию из шага 2. Что, если событие опоздало и пришло уже после закрытия его окна? Наивный вариант — выбросить его (потеря данных) или

выдать новый результат поверх старого (дашборд «мигает» промежуточным состоянием). miniFlink делает третье: атомарно *корректирует* результат.

Когда в уже закрытое окно приходит позднее значение, оператор эмитит `Update( $o \rightarrow n$ )` — одно событие, заменяющее старый результат o новым n без промежуточного состояния. Для потребителя это выглядит как мгновенный переход, без «мигания». Фундаментальный закон, который мы проверяем property-тестами для всех stateless-операторов:

$$\text{Update}(o \rightarrow n) \equiv \text{Retract}(o) \text{ затем } \text{Data}(n),$$

то есть атомарный `Update` — это в точности пара «отозвать старое, выдать новое», только без промежуточного шага. Писать обработку `Retract/Update` вручную обычно не нужно: операторы (`map`, `filter`, `window_agg`, `trigger`) делают это сами и согласованно.

10 Шаг 9. Персистентность без изменения пайплайна

Финальное требование: пережить перезапуск процесса. Активная тревога не должна исчезнуть из-за рестарта — иначе диспетчер увидит мнимое снятие.

Ключевая идея miniFlink — *ортогональная* персистентность: один и тот же пайплайн работает с durability и без неё, режим выбирается *снаружи*, а не вшит в операторы. Сначала пишем пайплайн как обычно:

```
let pipeline src =
  src
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Trigger.of_stream alert_spec
```

Без персистентности — просто запускаем. С персистентностью — оборачиваем тот же вызов в durable-контекст:

```
(* в памяти, ничего не сохраняется *)
let ephemeral () = pipeline event_stream

(* то же самое, но состояние durable *)
let durable () =
  let backend = Persistence_backend.of_memory (Hashtbl.create 16) in
  Runtime_context.with_context
    (Runtime_context.durable backend)
    (fun () -> pipeline event_stream)
```

Пайплайн не изменился ни на строку — разница только в обёртке `with_context`. Сериализация тоже ортогональна: состояние операторов сериализуется по умолчанию (через `Marshal`), так что для crash-recovery не нужно писать ни одного кодека; для эволюции схемы между деплоями можно задать реестр кодеков в контексте, опять же не трогая пайплайн. В продакшене вместо in-memory backend подставляется RocksDB; пайплайн при этом снова не меняется.

Это свойство — что durable-прогон с «крашем» посередине даёт тот же результат, что непрерывный — мы проверяем property-тестом на сотнях случайных потоков и точек краша.

11 Шаг 10. Масштабирование и exactly-once

Всё выше — однопоточный пайплайн. Для пропускной способности `Parallel.run_parallel_simple` шардирует поток по ключу на N воркеров. Когда нужна не просто параллельность, а

exactly-once с восстановлением после сбоя, используется `Checkpoint_parallel.run_exactly_once`: координатор инжектирует barrier-маркеры (снимок в стиле Chandy–Lamport), каждый воркер снапшотит своё состояние, а checkpoint фиксируется атомарно, когда снимки собраны со всех воркеров. Транзакционный sink и восстановление source по offset замыкают *exactly-once* на одном узле.

```
Checkpoint_parallel.run_exactly_once
~workers:4 ~capacity:256 ~checkpoint_every:1000
~key_of ~make_state ~process ~source ~sink ~store ()
```

Это другой, более высокий уровень гарантий, чем per-operator персистентность из шага 9: тот отвечает за восстановление состояния оператора, этот — за end-to-end *exactly-once* со стороны источника и приёмника. Выбирайте по задаче: для большинства пайплайнов достаточно ортогональной персистентности.

Замечание про параллелизм. На OCaml 4 истинной параллельности нет (runtime-lock), но конвейеризация через каналы прячет задержки; измеренное ускорение — 2.48× на 4 воркерах. На OCaml 5 с `Domain` появляется реальный параллелизм, и тот же код пользуется им без изменений.

12 Что дальше

Вы прошли путь от потока-функции до durable-пайплайна с тревогами. Дальнейшие шаги:

- **Примеры.** Каталог `examples/` (`ex01–ex10`) идёт по возрастанию сложности; `ex07` — полноценный сервис мониторинга шахты с триангуляцией и retract-пайплайном газовых тревог.
- **Справочник API.** Интерфейсные файлы `.mli` (особенно `pipe.mli` и `trigger.mli`) написаны как самостоятельные руководства с разбором краевых случаев.
- **Глубже в семантику.** Статья `miniflink_article` разбирает устройство движка; `docs/orthogonal-persistence.md` — модель персистентности; `docs/atomic-update-event.md` — семантику `Update`.
- **Прикладной разбор.** Статья по `ex07` (`docs/ex07_article`) показывает, как из этих операторов собирается реальная система безопасности.

Главная мысль, которую стоит унести: пайплайн `miniFlink` читается как спецификация. Ключ объявлен один раз, время и поздние данные обработаны операторами, а решения о персистентности и параллелизме приняты снаружи и не засоряют логику. Это и есть та декларативность, ради которой всё затевалось.